

Nätverkstrafik-undersökning

Namn: Mohamed Alomar

Datum: 23/24 september 2026

Miljö: Ubuntu i WSL

Del A – Miljö och metod

Jag gjorde uppgiften i Ubuntu i WSL. Jag använde tcpdump för att fånga nätverkstrafik och tshark för att läsa pcap-filen. Jag använde interfacet any. Fångsten var begränsad till högst 100 paket och pcap-filen innehöll 68 paket.

Jag testade trafik som jag själv skapade: ping mot localhost, en lokal HTTP-server och HTTPS mot example.com. Rå pcap ska sparas lokalt och inte läggas upp på GitHub.

Skillnaden mot lärarens demonstration är att jag använde WSL i stället för OCI. Därför kan interface, IP-adresser och nätverksväg se annorlunda ut.

Del B – Paketets väg

Jag tittade på HTTPS-trafiken till example.com. Först gjordes ett DNS-uppslag. Sedan skapades en TCP-anslutning till port 443. Efter TCP-handshaken startade TLS.

I min miljö var klientadressen privat och serveradressen publik. Default route gick via min WSL-gateway. Privat trafik kan behöva NAT/PAT när den går vidare ut från den lokala miljön. Exakt hur NAT görs går inte att se säkert bara från pcap-filen.

En brandvägg kan påverka trafiken innan den lämnar Linuxmiljön. En regel kan till exempel tillåta utgående TCP till port 443. Pcap-filen visar trafiken, men inte vilken brandväggsregel som faktiskt tillät den.

Del C – Protokoll

Protokoll	Evidens	Vad jag såg
DNS	Paket 35–38	Fråga om example.com och svar med IP-adresser.
ICMP	Paket 15–20	3 Echo Request och 3 Echo Reply mot localhost.
TCP	Paket 23–25 och 39–41	SYN, SYN/ACK och ACK. Anslutningarna startade normalt.
HTTP	Paket 26 och 30	GET / och svar 200. Innehållet var läsbart.
TLS/HTTPS	Paket 42–64	ClientHello, ServerHello och sedan krypterad trafik.

Alla fem protokoll finns i fångsten. Om jag bara hade sett delar av ett protokoll skulle jag behöva beskriva det som osäkert. Här finns däremot paket som stödjer varje del.

Del D – Två flöden

Flöde 1 – HTTP

Flödet gick mellan localhost och den lokala HTTP-servern på port 8000. Paket 23 var SYN, 24 var SYN/ACK och 25 var ACK. Sedan kom HTTP-data. Paket 26 visar GET / och paket 30 visar svar 200.

Detta var vad jag förväntade mig av en vanlig TCP- och HTTP-anslutning. Jag såg ingen tydlig timeout eller reset. En annan förklaring till ett problem i ett annat fall skulle kunna vara att servern inte lyssnar eller att trafik stoppas av en brandvägg. För en säkrare slutsats skulle jag även kunna kontrollera serverlogg och brandväggslogg.

Flöde 2 – HTTPS

HTTPS-flödet gick från min WSL-klient till en publik server på port 443. Paket 39 var SYN, 40 SYN/ACK och 41 ACK. Paket 42 var TLS ClientHello och paket 45 var TLS ServerHello. Efter det kom TLS-trafik.

Detta stämmer med hur en HTTPS-anslutning normalt startar. Jag såg ingen tydlig timeout eller reset i det analyserade flödet. Om kommunikationen hade avbrutits kunde andra förklaringar vara nätverksproblem, serverproblem eller brandvägg. För att veta mer skulle jag behöva fler loggar eller en längre fångst.

Del E – HTTP och HTTPS

	HTTP	TLS/HTTPS
Synlig metadata	IP, port och HTTP-information.	IP, port 443 och TLS-handshake.
Läsbar data	GET / och svar 200 kan läsas.	Själva applikationsinnehållet är krypterat.
Felsökning	Mycket av trafiken kan läsas direkt.	Handshake, adresser och paketflöde kan fortfarande analyseras.
Konfidentialitet	Innehållet kan exponeras.	Innehållet skyddas, men viss metadata syns.

Del F – Brandvägg och hardening

HTTPS-flödet använder utgående TCP på port 443. En brandvägg kan därför ha en regel som tillåter nödvändig utgående trafik på port 443. Den lokala HTTP-servern lyssnade på localhost:8000. Det betyder inte att port 8000 är öppen mot nätverket.

Default deny betyder att trafik som inte behövs ska nekas. Bara nödvändiga portar och tjänster bör tillåtas. Jag ändrade inte några brandväggsregler. De flöden jag själv skapade är rimliga för testmiljön eftersom HTTP-servern var lokal och HTTPS var utgående trafik.

Del G – CIA och evidens

Konfidentialitet: Pcap kan innehålla IP-adresser, DNS-information och okrypterad HTTP-data. Därför ska rå pcap skyddas. TLS skyddar själva innehållet bättre.

Integritet: Jag använde paketnummer och filnamnet network-investigation.pcap för att följa analysen. En SHA-256-hash kan användas lokalt för att kontrollera att filen inte ändrats.

Tillgänglighet: DNS fick svar, ICMP fick svar och TCP-handshakes lyckades. HTTPS fortsatte till TLS-handshake och krypterad trafik.

Begränsningar: Fångsten var liten och begränsad till 100 paket. Jag använde WSL och any, så miljön är inte exakt samma som OCI. Annan trafik kan ha missats och TLS-innehållet kan inte läsas direkt.

Del H – Slutsats

Jag såg DNS, ICMP, TCP, HTTP och HTTPS i min trafik. Den lokala HTTP-anslutningen fungerade och gav svar 200. HTTPS-anslutningen gick från TCP till TLS.

Det jag är säkrast på är det som syns direkt i pcap-filen, till exempel paketnummer, portar och TCP-flaggor. Jag kan däremot inte se exakt vad som finns inne i den krypterade TLS-trafiken.

Min rekommendation är att bara öppna portar som behövs och använda HTTPS när information ska skyddas. Som nästa steg skulle jag kunna jämföra pcap med brandväggs- och systemloggar.

AI-användning

Jag använde ChatGPT för att förstå uppgiften, få hjälp med kommandon och strukturera rapporten. Jag körde själv kommandona och kontrollerade resultaten mot min egen pcap. Paketnummer och tekniska slutsatser kommer från min egen analys. Jag använde inte förslag som inte stämde med min pcap.

Evidens

I originalarbetet finns skärmbilder för ping, tcpdump, protocol hierarchy, DNS, ICMP, TCP, HTTP och TLS. I den här versionen är känsliga adressuppgifter inte upprepade i rapporttexten. Rå pcap ska fortfarande inte publiceras.

Kort evidenslista

Evidens	Paket
DNS query/response	35–38
ICMP Echo Request/Reply	15–20
TCP handshake HTTP	23–25
HTTP GET/200	26 och 30
TCP/TLS handshake	39–45
TLS-trafik	42–64

```
mouhamadaloumar@5CD222 x + v
mouhamadaloumar@5CD2226SBH:~$ ping -c 3 127.0.0.1
PING 127.0.0.1 (127.0.0.1) 56(84) bytes of data.
64 bytes from 127.0.0.1: icmp_seq=1 ttl=64 time=0.063 ms
64 bytes from 127.0.0.1: icmp_seq=2 ttl=64 time=0.041 ms
64 bytes from 127.0.0.1: icmp_seq=3 ttl=64 time=0.064 ms

--- 127.0.0.1 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2031ms
rtt min/avg/max/mdev = 0.041/0.056/0.064/0.010 ms
mouhamadaloumar@5CD2226SBH:~$ |
```

Bild 2 – Tcpcdump startade

```
mouhamadaloumar@5CD222 x + v
mouhamadaloumar@5CD2226SBH:~$ sudo tcpdump -i any -c 100 -w ~/network-investigation.pcap
[sudo: authenticate] Password:
sudo: Authentication failed, try again.
[sudo: authenticate] Password:
tcpdump: WARNING: any: That device doesn't support promiscuous mode
(Promiscuous mode not supported on the "any" device)
tcpdump: listening on any, link-type LINUX_SLL2 (Linux cooked v2), snapshot length 262144 bytes
|
```

Bild 3 – Protocol Hierarchy Statistics

```
:1.5em}div{opacity:0.8}a:link,a:visited{color:#348}</style></head><body><div><h1>Example Domain</h1><p>This domain is for use in d
mentation examples without needing permission. Avoid use in operations.</p><p><a href="https://iana.org/domains/example">Learn mo
a></p></div></body></html>
mouhamadaloumar@5CD2226SBH:~$ sudo pkill tcpdump
mouhamadaloumar@5CD2226SBH:~$ ls -lh ~/network-investigation.pcap
-rw-r--r-- 1 tcpdump tcpdump 18K Sep 23 15:37 /home/mouhamadaloumar/network-investigation.pcap
[1]- Done sudo tcpdump -i any -c 100 -w ~/network-investigation.pcap > /tmp/tcpdump.log 2>&1
mouhamadaloumar@5CD2226SBH:~$ tshark -r ~/network-investigation.pcap -q -z io,phs

=====
Protocol Hierarchy Statistics
Filter:

frame                                frames:68 bytes:16690
  sll                                frames:68 bytes:16690
    ip                                frames:62 bytes:16402
      udp                             frames:16 bytes:3228
        ntp                           frames:12 bytes:2832
          dns                         frames:4 bytes:396
            icmp                      frames:6 bytes:624
              tcp                     frames:40 bytes:12550
                http                  frames:2 bytes:1294
                  data-text-lines    frames:1 bytes:1144
                    tls               frames:13 bytes:9272
                      arp             frames:6 bytes:288
=====
mouhamadaloumar@5CD2226SBH:~$ |
```

Bild 4 – DNS query och response

```
mouhamadaLoumar@5CD2226SBH:~$ tshark -r ~/network-investigation.pcap -Y dns -T fields -e frame.number -e ip.src -e ip.dst -e dns.qry.
name -e dns.a -e dns.aaaa
35 10.255.255.254 10.255.255.254 example.com
36 10.255.255.254 10.255.255.254 example.com
37 10.255.255.254 10.255.255.254 example.com 104.20.23.154,172.66.147.243
38 10.255.255.254 10.255.255.254 example.com 2606:4700:10::ac42:93f3,2606:4700:10::6814:179a
mouhamadaLoumar@5CD2226SBH:~$
```

Bild 5 – ICMP Echo Request/Reply

```
mouhamadaLoumar@5CD2226SBH:~$ tshark -r ~/network-investigation.pcap -Y dns -T fields -e frame.number -e ip.src -e ip.dst -e dns.qry.
name -e dns.a -e dns.aaaa
35 10.255.255.254 10.255.255.254 example.com
36 10.255.255.254 10.255.255.254 example.com
37 10.255.255.254 10.255.255.254 example.com 104.20.23.154,172.66.147.243
38 10.255.255.254 10.255.255.254 example.com 2606:4700:10::ac42:93f3,2606:4700:10::6814:179a
mouhamadaLoumar@5CD2226SBH:~$ tshark -r ~/network-investigation.pcap -Y icmp -T fields -e frame.number -e ip.src -e ip.dst -e icmp.type -e icmp.seq
15 127.0.0.1 127.0.0.1 8 1
16 127.0.0.1 127.0.0.1 0 1
17 127.0.0.1 127.0.0.1 8 2
18 127.0.0.1 127.0.0.1 0 2
19 127.0.0.1 127.0.0.1 8 3
20 127.0.0.1 127.0.0.1 0 3
mouhamadaLoumar@5CD2226SBH:~$
```

Bild 6 – TCP SYN/SYN-ACK

```
mouhamadaLoumar@5CD2226SBH:~$ tshark -r ~/network-investigation.pcap -Y "tcp.flags.syn==1" -l fields -e frame.number -e ip.src -e tcp.s
srcport -e ip.dst -e tcp.dstport -e tcp.flags
21 127.0.0.1 55784 127.0.0.1 8000 0x0002
23 127.0.0.1 44330 127.0.0.1 8000 0x0002
24 127.0.0.1 8000 127.0.0.1 44330 0x0012
39 172.21.109.103 52888 104.20.23.154 443 0x0002
40 104.20.23.154 443 172.21.109.103 52888 0x0012
mouhamadaLoumar@5CD2226SBH:~$
```

Bild 7 – Komplet TCP handshake

```
mouhamadaLoumar@5CD2226SBH:~$ tshark -r ~/network-investigation.pcap -Y "tcp.stream==1" -T fields -e frame.number -e ip.src -e tcp.s
srcport -e ip.dst -e tcp.dstport -e tcp.flags
23 127.0.0.1 44330 127.0.0.1 8000 0x0002
24 127.0.0.1 8000 127.0.0.1 44330 0x0012
25 127.0.0.1 44330 127.0.0.1 8000 0x0010
26 127.0.0.1 44330 127.0.0.1 8000 0x0018
27 127.0.0.1 8000 127.0.0.1 44330 0x0010
28 127.0.0.1 8000 127.0.0.1 44330 0x0018
29 127.0.0.1 44330 127.0.0.1 8000 0x0010
30 127.0.0.1 8000 127.0.0.1 44330 0x0018
31 127.0.0.1 44330 127.0.0.1 8000 0x0010
32 127.0.0.1 8000 127.0.0.1 44330 0x0011
33 127.0.0.1 44330 127.0.0.1 8000 0x0011
34 127.0.0.1 8000 127.0.0.1 44330 0x0010
mouhamadaLoumar@5CD2226SBH:~$
```

Bild 8 – HTTP request/response

```
mouhamadaLoumar@5CD2226SBH:~$ tshark -r ~/network-investigation.pcap -Y http -l fields -e frame.number -e ip.src -e tcp.srcport -e ip
dst -e tcp.dstport -e http.request.method -e http.request.uri -e http.response.code
26 127.0.0.1 44330 127.0.0.1 8000 GET /
30 127.0.0.1 8000 127.0.0.1 44330 / 200
mouhamadaLoumar@5CD2226SBH:~$
```

Bild 9 – TLS ClientHello/ServerHello

```
mouhamadaLoumar@5CD2226SBH:~$ tshark -r ~/network-investigation.pcap -Y tls -l fields -e frame.number -e ip.src -e tcp.srcport -e ip
dst -e tcp.dstport -e tls.record.content.type -e tls.handshake.type
42 172.21.109.103 52888 104.20.23.154 443 22 1
45 104.20.23.154 443 172.21.109.103 52888 22,20 2
46 104.20.23.154 443 172.21.109.103 52888
49 172.21.109.103 52888 104.20.23.154 443 20
50 172.21.109.103 52888 104.20.23.154 443
51 172.21.109.103 52888 104.20.23.154 443
53 104.20.23.154 443 172.21.109.103 52888
54 172.21.109.103 52888 104.20.23.154 443
55 104.20.23.154 443 172.21.109.103 52888
56 104.20.23.154 443 172.21.109.103 52888
57 104.20.23.154 443 172.21.109.103 52888
59 172.21.109.103 52888 104.20.23.154 443
60 172.21.109.103 52888 104.20.23.154 443
mouhamadaLoumar@5CD2226SBH:~$
```

Bild 10 – TCP/TLS-flöde över port 443

```
mouhamadaLoumar@SCD2226SBH:~$ tshark -r ~/network-investigation.pcap -Y "tcp.port==443" -l fields -e frame.number -e ip.src -e tcp.s
cport -e ip.dst -e tcp.dstport -e tcp.flags -e tls.handshake.type
39 172.21.109.103 52888 104.20.23.154 443 0x0002
40 104.20.23.154 443 172.21.109.103 52888 0x0012
41 172.21.109.103 52888 104.20.23.154 443 0x0010
42 172.21.109.103 52888 104.20.23.154 443 0x0018 1
43 104.20.23.154 443 172.21.109.103 52888 0x0010
44 104.20.23.154 443 172.21.109.103 52888 0x0010
45 104.20.23.154 443 172.21.109.103 52888 0x0010 2
46 104.20.23.154 443 172.21.109.103 52888 0x0018
47 172.21.109.103 52888 104.20.23.154 443 0x0010
48 172.21.109.103 52888 104.20.23.154 443 0x0010
49 172.21.109.103 52888 104.20.23.154 443 0x0018
50 172.21.109.103 52888 104.20.23.154 443 0x0018
51 172.21.109.103 52888 104.20.23.154 443 0x0018
52 104.20.23.154 443 172.21.109.103 52888 0x0010
53 104.20.23.154 443 172.21.109.103 52888 0x0018
54 172.21.109.103 52888 104.20.23.154 443 0x0018
55 104.20.23.154 443 172.21.109.103 52888 0x0018
56 104.20.23.154 443 172.21.109.103 52888 0x0018
57 104.20.23.154 443 172.21.109.103 52888 0x0018
58 172.21.109.103 52888 104.20.23.154 443 0x0010
59 172.21.109.103 52888 104.20.23.154 443 0x0018
60 172.21.109.103 52888 104.20.23.154 443 0x0018
61 172.21.109.103 52888 104.20.23.154 443 0x0011
62 104.20.23.154 443 172.21.109.103 52888 0x0010
63 104.20.23.154 443 172.21.109.103 52888 0x0011
64 172.21.109.103 52888 104.20.23.154 443 0x0010
mouhamadaLoumar@SCD2226SBH:~$
```